

Document number	P2730R0
Date	2022-11-27
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	Evolution Working Group (EWG)

variable scope

Table of contents

- variable scope
 - Changelog
 - Abstract
 - Motivation
 - Proposed Wording
 - Details
 - Other Anonymous Things
 - Value Categories
 - Summary
 - Frequently Asked Questions
 - References

Changelog

R0

- The variable scope content was extracted and merged from the `temporary storage class specifiers` ^[1] and `implicit constant initialization` ^[2] proposals.

Abstract

This paper proposes the standard changes the default lifetime of temporaries from being statement scope to block and variable scope. These two changes would eliminate immediate dangling of the stack. It would also fix **direct** dangling that occurs in the body of a function. By

handling the **indirect** returning of locals passed as arguments to functions, this proposal would also compliment `simpler implicit move` ^[3], which resolved direct returning of local. This would leave `c++` in a good position to identify the remaining indirect dangling with help of return/parameter dependency attributes from the programmer. ^[4]

Motivation

There are multiple resolutions to dangling in the `c++` language.

1. Produce an error
 - `Simpler implicit move` ^[3:1]
2. Fix with block/variable scoping
 - `Fix the range-based for loop, Rev2` ^[5]
 - `Get Fix of Broken Range-based for Loop Finally Done` ^[6]
 - **This proposal**
3. Fix by making the instance global

All are valid resolutions and individually are better than the others, given the scenario. This proposal is focused on the second option, which is to fix by changing the scope of the temporary instance from statement scope to block/variable scope.

Dangling the stack is shocking because it violates our trust in our compilers and language, since they are primarily responsible for the stack. However, there are three types of dangling that are even more shocking than the rest.

1. Returning a **direct** reference to a local
 - partially resolved by `Simpler implicit move` ^[3:2]
2. **Immediate dangling**
3. Dangling Constants

This proposal is primarily focused on immediate dangling. Returning a reference that was passed as an argument to a function is perfectly legitimate.

```
const std::string& f(const std::string& defaultValue)
{
    return defaultValue;
}

const std::string& value = f("Hello World"s); // immediate dangling
// using `value` is bad
```

The function `f` does not know whether `defaultValue` was created locally, globally or dynamically, nor does it care. The function `f` does expect that whatever instance is passed to it is alive after `f` is called. This makes sense because globals live to the end of the program, dynamically allocated instances provided would need to be deleted afterward and locals provided are lower on the stack since they were created in the caller's scope and live to the end of the block they were allocated in. This is reaffirmed in a note in the `C++ Core Guidelines` .

C++ Core Guidelines
F.42: Return a `T*` to indicate a position (only) ^[7]

...

Note *Do not return a pointer to something that is not in the caller's scope; see F.43.* ^[8]

Temporaries default lifetime is something totally different.

“6.7.7 Temporary objects [class.temporary]”

“(6.12) — A temporary bound to a reference in a new-initializer (7.6.2.8) persists until the completion of the full-expression containing the new-initializer.”

“[Note 7: This might introduce a dangling reference. — end note]”

For simplicity sake, I'll call this statement `scope[d]` . As note 7 admits, this introduces more dangling to the language, besides the existing dangling of returning a reference to a local. It is also very unnatural in multiple ways.

indirect dangling of caller's local

What is `c++` doing when calling a function such as `f` in the previous example?

Programmer's Code	What C++ did!
<pre>{ f("Hello World"s); }</pre>	<pre>{ { auto anonymous = "Hello World"s; f(anonymous); } }</pre>

When dangling occurs, what programmers usually do is name the instance in order to fix it. Programmers don't do this initially because this increases both superfluous names and lines of code which only increases the complexity of the program. A programmer expects the compiler to write code better than a programmer would, not worse. Dangling is way worse. In this case, the compiler is adding undersirable braces. When has a c or c++ developer ever polluted every function call in their program with superfluous braces. This is just unnatural. Unfortunately, things get worse. What does c++ do when that same function returns?

Programmer's Code	What C++ did!
<pre>{ const std::string& value = f("Hello World"s); }</pre>	<pre>{ const std::string& v { auto anonymous = value = f(anonym } }</pre>

In this case, the compiler wrote code that is impossible for the programmer to write. The compiler scoped the arguments without scoping the return. A c++ programmer can't write a brace scope and have it apply to some locals declared in it and not to others. This makes things harder to understand.

outer scope		<pre>// before the call of the function `f`</pre>
	`value` scope	<pre>const std::string& value = f("Hello World"s);</pre>
		<pre>// `value` dangles for the remainder of the outer scope</pre>

The result is the unnatural indirect dangling of caller's local . This is also unnatural for any programmer with a c background.

“6.5.2.5 Compound literals”

paragraph 5

*“The value of the compound literal is that of an unnamed object initialized by the initializer list. If the compound literal occurs outside the body of a function, the object has static storage duration; otherwise, it has **automatic storage duration associated with the enclosing block.**”*

For simplicity sake, we’ll call this `block scope`. In short, `c` has less dangling than `c++`. Dropping the superfluous unnatural braces fixes `indirect dangling of caller's local` but it also fixes `immediate dangling`.

immediate dangling

```
std::string_view sv = "hello world"s; // immediate dangling reference
```

Set aside the fact that this should be fixed by constant initialization. This example dangles similar to `indirect dangling of caller's local`. However, something else unnatural is happening.

outer scope	<code>`sv` scope</code>	<code>// before assigning `sv`</code>
		<code>std::string_view sv = "hello world"s;</code>
		<code>// `sv` dangles for the remainder of the outer scope</code>

When initializing or reinitializing locals, the lifetime of the temporary is divorced from the lifetime of the local variable that it is being assigned too. The temporary `"hello world"s` should have died when the variable `sv` died, not sooner. `block scope` would have fixed it in this instance but it doesn’t work with all locals as the next example illustrates.

```

std::string_view sv = "initial value"s;
if(randomBool())
{
    sv = "if value"s;
}
else
{
    sv = "else value"s;
}

```

While `block scope` would fix the `"initial value"s` temporary, it would not fix the `"if value"s` or the `"else value"s` temporaries. In all three cases, the temporary was divorced from the lifetime of the local variable `sv`, that it is being assigned to. Had these temporaries died when the assigned variable `sv` died then there would not be any dangling. For simplicity sake, we'll call this `variable scope`. `variable scope` fixes all the non return **direct** dangling that occurs in the body of a function. `variable scope` does not fix dangling associated with it being arguments to a function call as the variable in question is the parameter/argument which would just make it `statement scope`.

Let me summarize, statement scoping of a temporary is unnatural because ...

- It prevents/discourages functions from returning references/pointers to instances that do exist lower on the stack.
- It introduces superfluous scopes that the programmer would never code.
- It introduces code that the programmer could never write thus making things harder to understand.
- The lifetime of the temporary is divorced from the lifetime of the local variable that it is being assigned to.
- It creates all of the more complex direct dangling associated with parameters/arguments as well as what exists in the body of the function.

Consequently, all of the non return direct dangling can be fixed by doing the following two things.

- local variable's temporaries get `variable scope` and
- function argument's temporaries get `block scope`

Extending the lifetime of the temporary to be the lifetime of the variable to which it is assigned is not unreasonable for C++. Matter of fact it is already happening but the rules are so targeted that it limits its use by many programmers as the following examples illustrate.

“6.7.7 Temporary objects”

...

“₅ There are ... contexts in which temporaries are destroyed at a different point than the end of the full expression.”

...

“(6.8)”

```
template<typename T> using id = T;

int i = 1;
int&& a = id<int[3]>{1, 2, 3}[i]; // temporary array has same lifetime as a
const int& b = static_cast<const int&>(0); // temporary int has same lifetime as b
int&& c = cond ? id<int[3]>{1, 2, 3}[i] : static_cast<int&&>(0);
// exactly one of the two temporaries is lifetime-extended
```



```
const int& x = (const int&)1; // temporary for value 1 has same lifetime as x
```

```
struct S {
    const int& m;
};
const S& s = S{1}; // both S and int temporaries have lifetime of s
```

Even though the standard says that the temporaries lifetime is extended to match that of the variable, in reality, it is the same as `block scope`. The one example that is closest to the `if/else` variable scope example is as follows:

```
int&& c = cond ? id<int[3]>{1, 2, 3}[i] : static_cast<int&&>(0);
// exactly one of the two temporaries is lifetime-extended
```

This isn't quite the same as both temporaries being `lifetime-extended`. Shouldn't the `?:` ternary operator example work for variables that are delayed initialized or reinitialized inside of the `if` and `else` clauses of `if/else` statements? Shouldn't the block of the variable declared above a `if/else` statement be used over the blocks of `if` and `else` clause where the temporary was assigned? This seems reasonable and consequently `variable scope` fixes even more dangling than `block scope`.

```

std::string_view sv = "1"s;
if(randomBool())
{
    switch(randomInt())
    {
        case 1:
            sv = "10"s;
            break;
        default:
            sv = "100"s;
            break;
    }
}
else
{
    if(randomBool())
    {
        sv = "1000"s;
    }
    else
    {
        sv = "10000"s;
    }
}

```

Here, variable scope fixes all 5 dangling references. It works regardless of the nesting and it also works with other control structures such as `switch/case`.

towards a less complicated standard

Changing the lifetime of temporaries from `statement scope` to `block scope` and `variable scope` would also vastly simplify the standard, making dangling even easier to teach. Notice the levels of complexity when comparing `c` and the as proposed verbiage with the current standard.

C verbiage

“6.5.2.5 Compound literals”

paragraph 5

*“The value of the compound literal is that of an unnamed object initialized by the initializer list. If the compound literal occurs outside the body of a function, the object has static storage duration; otherwise, it has **automatic storage duration associated with the enclosing block.**”*

As proposed

6.7.7 Temporary objects

...

⁴ When an implementation introduces a temporary object of a class that has a non-trivial constructor (11.4.5.2, 11.4.5.3), it shall ensure that a constructor is called for the temporary object. Similarly, the destructor shall be called for a temporary with a non-trivial destructor (11.4.7). Temporary objects are destroyed via automatic storage duration (6.7.5.4) associated with the enclosing block of the expression as if the compiler was naming the temporaries anonymously or via automatic storage duration associated with the enclosing block of the variable to which the temporary is assigned, whichever is greater lifetime.

C++ verbiage

6.7.7 Temporary objects

...

⁴ When an implementation introduces a temporary object of a class that has a non-trivial constructor (11.4.5.2, 11.4.5.3), it shall ensure that a constructor is called for the temporary object. Similarly, the destructor shall be called for a temporary with a non-trivial destructor (11.4.7). Temporary objects are destroyed as the last step in evaluating the full-expression (6.9.1) that (lexically) contains the point where they were created. This is true even if that evaluation ends throwing an exception. The value computations and side effects of destroying a temporary object are associated only with the full-expression, not with any specific subexpression.

⁵ There are three contexts in which temporaries are destroyed at a different point than the end of the full expression. The first context is when a default constructor is called to initialize an element of an array with no corresponding initializer (9.4). The second context is when a copy constructor is called to copy an element of an array while the entire array is copied (7.5.5.3, 11.4.5.3). In each case, if the constructor has one or more default arguments, the destruction of every temporary created in a default argument is sequenced before the construction of the next array element, if any.

⁶ The third context is when a reference binds to a temporary object.²⁹ The temporary object to which the reference is bound or the temporary object that is the complete object of a subobject to which the reference is bound persists for the lifetime of the reference if the glvalue to which the reference is bound was obtained through one of the following:

(6.1) — a temporary materialization conversion (7.3.5),

(6.2) — (expression), where expression is one of these expressions,

(6.3) — subscripting (7.6.1.2) of an array operand, where that operand is one of these expressions

(6.4) — a class member access (7.6.1.5) using the . operator where the left operand is one of these expressions and the right operand designates a non-static data member of non-reference type

(6.5) — a pointer-to-member operation (7.6.4) using the .* operator where the left operand is one of these expressions and the right operand is a pointer to data member of non-reference type,

(6.6) — a

(6.6.1) — `const_cast` (7.6.1.11),

(6.6.2) — `static_cast` (7.6.1.9),

(6.6.3) — `dynamic_cast` (7.6.1.7), or

(6.6.4) — reinterpret_cast (7.6.1.10)

converting, without a user-defined conversion, a glvalue operand that is one of these expressions to a glvalue that refers to the object designated by the operand, or to its complete object or a subobject thereof,

(6.7) — a conditional expression (7.6.16) that is a glvalue where the second or third operand is one of these expressions, or

(6.8) — a comma expression (7.6.20) that is a glvalue where the right operand is one of these expressions.

[Example 2:

```
template<typename T> using id = T;

int i = 1;
int&& a = id<int[3]>{1, 2, 3}[i]; // temporary array has same lifetime as a
const int& b = static_cast<const int&>(0); // temporary int has same lifetime as
int&& c = cond ? id<int[3]>{1, 2, 3}[i] : static_cast<int&&>(0);
// exactly one of the two temporaries is lifetime-extended
```

— end example]

[Note 5: An explicit type conversion (7.6.1.4, 7.6.3) is interpreted as a sequence of elementary casts, covered above.

[Example 3:

```
const int& x = (const int&)1; // temporary for value 1 has same lifetime as x
```

— end example]

— end note]

[Note 6: If a temporary object has a reference member initialized by another temporary object, lifetime extension applies recursively to such a member's initializer.

[Example 4:

```
struct S {  
    const int& m;  
};  
const S& s = S{1}; // both S and int temporaries have lifetime of s
```

— end example]

— end note]

The exceptions to this lifetime rule are:~

(6.9) — A temporary object bound to a reference parameter in a function call (7.6.1.3) persists until the completion of the full-expression containing the call.

(6.10) — A temporary object bound to a reference element of an aggregate of class type initialized from a parenthesized expression-list (9.4) persists until the completion of the full-expression containing the expression-list.

(6.11) — The lifetime of a temporary bound to the returned value in a function return statement (8.7.4) is not extended; the temporary is destroyed at the end of the full-expression in the return statement.

(6.12) — A temporary bound to a reference in a new-initializer (7.6.2.8) persists until the completion of the full-expression containing the new-initializer.

[Note 7: This might introduce a dangling reference. — end note]

[Example 5:

```
struct S { int m; const std::pair<int,int>& mp; };  
S a { 1, {2,3} };  
S* p = new S{ 1, {2,3} }; // creates dangling reference
```

— end example]

7 The destruction of a temporary whose lifetime is not extended by being bound to a reference is sequenced before the destruction of every temporary which is constructed earlier in the same expression. If the lifetime of two or more temporaries to which references are bound ends at the same point, these temporaries are destroyed at that point in the reverse order of the completion of their construction. In addition, the destruction of temporaries bound to references shall take into account the ordering of destruction of objects with static, thread, or automatic storage duration (6.7.5.2, 6.7.5.3, 6.7.5.4); that is, if obj1 is an object with the same storage duration as the

temporary and created before the temporary is created the temporary shall be destroyed before obj1 is destroyed; if obj2 is an object with the same storage duration as the temporary and created after the temporary is created the temporary shall be destroyed after obj2 is destroyed.

8 [Example 6:

```
struct S {  
    S();  
    S(int);  
    friend S operator+(const S&, const S&);  
    ~S();  
};  
S obj1;  
const S& cr = S(16)+S(23);  
S obj2;
```

The expression `S(16) + S(23)` creates three temporaries: a first temporary T1 to hold the result of the expression `S(16)`, a second temporary T2 to hold the result of the expression `S(23)`, and a third temporary T3 to hold the result of the addition of these two expressions. The temporary T3 is then bound to the reference `cr`. It is unspecified whether T1 or T2 is created first. On an implementation where T1 is created before T2, T2 shall be destroyed before T1. The temporaries T1 and T2 are bound to the reference parameters of `operator+`; these temporaries are destroyed at the end of the full-expression containing the call to `operator+`. The temporary T3 bound to the reference `cr` is destroyed at the end of `cr`'s lifetime, that is, at the end of the program. In addition, the order in which T3 is destroyed takes into account the destruction order of other objects with static storage duration. That is, because `obj1` is constructed before T3, and T3 is constructed before `obj2`, `obj2` shall be destroyed before T3, and T3 shall be destroyed before `obj1`.

— end example]

Why the complexity? The statement scope itself is simple, weighing in as just the first cited paragraph. However, all the other verbiage is for all of the, consistently the same, exceptions to the rule. This list is growing. This makes teaching C++ more difficult. This makes teaching dangling and dangling resolution more difficult because one must know whether dangling was even an issue or not due to whether the temporary was lifetime extended or not.

The question is, can the proposed wording cover the use cases in those numerous exceptions to the statement scoping rule. I have previously covered that the current standard says that all of these exceptions were scoped to the variable, while in fact were actually blocked scope. However, let's consider a recent addition in greater detail. That is the range based for loop dangling fix.

- Fix the range-based for loop, Rev2 [5:1]
- Get Fix of Broken Range-based for Loop Finally Done [6:1]

```
for (auto elem : foo().bar().getValues()) ...
```

“should be expanded equivalent to the following:” [5:2]

```
auto&& tmp1 = foo(); // lifetime of all temporaries extended
auto&& tmp2 = tmp1.bar(); // lifetime of all temporaries extended
auto&& rg = tmp2.getValues();
auto pos = rg.begin();
auto end = rg.end();
for ( ; pos != end; ++pos ) {
    auto elem = *pos;
    ...
}
```

In the wording of these proposals, it says that the temporaries persist until the completion of the range based for statement. In other words, the previous example should be revised to show the additional scope.

```
{
    auto&& tmp1 = foo(); // lifetime of all temporaries extended
    auto&& tmp2 = tmp1.bar(); // lifetime of all temporaries extended
    auto&& rg = tmp2.getValues();
    auto pos = rg.begin();
    auto end = rg.end();
    for ( ; pos != end; ++pos ) {
        auto elem = *pos;
        ...
    }
}
```

This is nothing more than `block scope` where the block in question is the additional block added via the revisions to the range based for loop. Notice too that each temporary was promoted to being an anonymously named variable. In short, this proposal's simplified verbiage covers all existing lifetime extensions.

The only real concern is what would this proposal break. Instances still get destroyed in a deterministic RAII fashion, however, temporary instances will live longer in order to fix dangling. The biggest category of types impacted by this would be by **lock** objects. However, this only occurs when locks are used as a temporary instead of recommended usage of a named

instance. This was the same problem for the `range based for loop fix` and was set aside for the greater good of fixing dangling defects in the `c++` language. The rationale is recorded in the following sections in that paper.

- Is there code that might be broken with the `fix`? [5:3]
- So, how much code is broken in practice? [5:4]
- What are the drawbacks of such a `fix`? [5:5]

That paper also answers these questions.

- Is there a performance penalty? [5:6]
- But don't we break the zero-overhead principle? [5:7]
- But don't we have tools to detect such lifetime problems? [5:8]
- Shouldn't we fix lifetime extension for references in general? [5:9]

Even if it is decided not to fix dangling by changing the default automatically, the paper `temporary storage class specifiers` [1:1] does propose allowing programmers to opt in to this change via a module level attribute. This would allow the vast majority of code bases to do away with most dangling and allow those who need more time to adjust to have the `statement scope` default. It would also serve as a vehicle to allow the standard to change over time. Granted this paper is all about fixing dangling as it is the best option and is inline with past decisions made for the `range based for loop fix`.

Proposed Wording

6.7.7 Temporary objects

...

4 When an implementation introduces a temporary object of a class that has a non-trivial constructor (11.4.5.2, 11.4.5.3), it shall ensure that a constructor is called for the temporary object. Similarly, the destructor shall be called for a temporary with a non-trivial destructor (11.4.7). Temporary objects are destroyed via automatic storage duration (6.7.5.4) associated with the enclosing block of the expression as if the compiler was naming the temporaries anonymously or via automatic storage duration associated with the enclosing block of the variable to which the temporary is assigned, whichever is greater lifetime. ~~as the last step in evaluating the full expression (6.9.1) that (lexically) contains the point where they were created. This is true even if that evaluation ends in throwing an exception. The value computations and side effects of destroying a temporary object are associated only with the full expression, not with any specific subexpression.~~

~~5 There are three contexts in which temporaries are destroyed at a different point than the end of the full expression. The first context is when a default constructor is called to initialize an element of an array with no corresponding initializer (9.4). The second context is when a copy constructor is called to copy an element of an array while the entire array is copied (7.5.5.3, 11.4.5.3). In either case, if the constructor has one or more default arguments, the destruction of every temporary created in a default argument is sequenced before the construction of the next array element, if any.~~

~~6 The third context is when a reference binds to a temporary object.²⁹ The temporary object to which the reference is bound or the temporary object that is the complete object of a subobject to which the reference is bound persists for the lifetime of the reference if the glvalue to which the reference is bound was obtained through one of the following:~~

~~(6.1) — a temporary materialization conversion (7.3.5);~~

~~(6.2) — (expression), where expression is one of these expressions;~~

~~(6.3) — subscripting (7.6.1.2) of an array operand, where that operand is one of these expressions;~~

~~(6.4) — a class member access (7.6.1.5) using the . operator where the left operand is one of these expressions and the right operand designates a non-static data member of non-reference type;~~

~~(6.5) — a pointer to member operation (7.6.4) using the .* operator where the left operand is one of these expressions and the right operand is a pointer to data member of non-reference type;~~

~~(6.6) — a~~

~~(6.6.1) — const_cast (7.6.1.11);~~

~~(6.6.2) — static_cast (7.6.1.9);~~

~~(6.6.3) — dynamic_cast (7.6.1.7); or~~

~~(6.6.4) — reinterpret_cast (7.6.1.10)~~

~~converting, without a user-defined conversion, a glvalue operand that is one of these expressions to a glvalue that refers to the object designated by the operand, or to its complete object or a subobject thereof;~~

~~(6.7) — a conditional expression (7.6.16) that is a glvalue where the second or third operand is one of these expressions; or~~

~~(6.8) — a comma expression (7.6.20) that is a glvalue where the right operand is one of these expressions.~~

~~{Example 2:~~

```
template<typename T> using id = T;

int i = 1;
int&& a = id<int[3]>{1, 2, 3}[i]; // temporary array has same lifetime as a
const int& b = static_cast<const int&>(0); // temporary int has same lifetime as b
int&& c = cond ? id<int[3]>{1, 2, 3}[i] : static_cast<int&&>(0);
// exactly one of the two temporaries is lifetime extended
```

~~—end example}~~

~~{Note 5: An explicit type conversion (7.6.1.4, 7.6.3) is interpreted as a sequence of elementary casts, covered above.~~

~~{Example 3:~~

```
const int& x = (const int&)1; // temporary for value 1 has same lifetime as x
```

~~—end example}~~

~~—end note}~~

~~{Note 6: If a temporary object has a reference member initialized by another temporary object, lifetime extension applies recursively to such a member's initializer.~~

~~{Example 4:~~

```
struct S {
—const int& m;
};
const S& s = S{1}; // both S and int temporaries have lifetime of s
```

~~—end example}~~

~~—end note}~~

~~The exceptions to this lifetime rule are:~~

~~(6.9) — A temporary object bound to a reference parameter in a function call (7.6.1.3) persists until the completion of the full expression containing the call.~~

~~(6.10) — A temporary object bound to a reference element of an aggregate of class type initialized from a parenthesized expression list (9.4) persists until the completion of the full expression containing the expression list.~~

~~(6.11) — The lifetime of a temporary bound to the returned value in a function return statement (8.7.4) is not extended; the temporary is destroyed at the end of the full expression in the return statement.~~

~~(6.12) — A temporary bound to a reference in a new initializer (7.6.2.8) persists until the completion of the full expression containing the new initializer.~~

~~{Note 7: This might introduce a dangling reference. — end note}~~

~~{Example 5:~~

```
struct S { int mi; const std::pair<int,int>& mp; };  
S a { 1, {2,3} };  
S* p = new S{ 1, {2,3} }; // creates dangling reference
```

~~— end example}~~

~~z The destruction of a temporary whose lifetime is not extended by being bound to a reference is sequenced before the destruction of every temporary which is constructed earlier in the same full expression. If the lifetime of two or more temporaries to which references are bound ends at the same point, these temporaries are destroyed at that point in the reverse order of the completion of their construction. In addition, the destruction of temporaries bound to references shall take into account the ordering of destruction of objects with static, thread, or automatic storage duration (6.7.5.2, 6.7.5.3, 6.7.5.4); that is, if obj1 is an object with the same storage duration as the temporary and created before the temporary is created the temporary shall be destroyed before obj1 is destroyed; if obj2 is an object with the same storage duration as the temporary and created after the temporary is created the temporary shall be destroyed after obj2 is destroyed.~~

~~g {Example 6:~~

```

struct S {
— S();
— S(int);
— friend S operator+(const S&, const S&);
— ~S();
};
S obj1;
const S& cr = S(16)+S(23);
S obj2;

```

The expression `S(16) + S(23)` creates three temporaries: a first temporary T1 to hold the result of the expression `S(16)`, a second temporary T2 to hold the result of the expression `S(23)`, and a third temporary T3 to hold the result of the addition of these two expressions. The temporary T3 is then bound to the reference `cr`. It is unspecified whether T1 or T2 is created first. On an implementation where T1 is created before T2, T2 shall be destroyed before T1. The temporaries T1 and T2 are bound to the reference parameters of `operator+`; these temporaries are destroyed at the end of the full expression containing the call to `operator+`. The temporary T3 bound to the reference `cr` is destroyed at the end of `cr`'s lifetime, that is, at the end of the program. In addition, the order in which T3 is destroyed takes into account the destruction order of other objects with static storage duration. That is, because `obj1` is constructed before T3, and T3 is constructed before `obj2`, `obj2` shall be destroyed before T3, and T3 shall be destroyed before `obj1`.

—end example}

...

NOTE: Wording still need to capture that these temporaries are no longer temporaries and that their value category is `lvalue`.

Details

Other Anonymous Things

The pain of immediate dangling associated with temporaries are especially felt when working with other anonymous language features of `C++` such as lambda functions.

Lambda functions

Whenever a lambda function captures a reference to a temporary it immediately dangles before an opportunity is given to call it, unless it is a immediately invoked lambda/function expression.

```

 [&c1 = "hello"s](const std::string& s)// OK
 {
     return c1 + " "s + s;
 }("world"s); // immediately invoked lambda/function expression

 auto lambda = [&c1 = "hello"s](const std::string& s) // immediate dangling
 {
     return c1 + " "s + s;
 }
 // ...
 lambda("world"s);

```

This problem is resolved when the scope of temporaries is to the enclosing block instead of the containing expression. This is the same had the temporary been named.

```

 auto anonymous = "hello"s;
 auto lambda = [&c1 = anonymous](const std::string& s)
 {
     return c1 + " "s + s;
 }
 // ...
 lambda("world"s);

```

Unless resolved, this problem will continue to be a problem for future c++ features, type erased or not, anonymous or not, where there are a separate construction and execution steps.

Value Categories

If temporaries can be changed to have block scope, variable scope or global scope than how does it affect their value categories? Currently, if the literal is a string than it is a `lvalue` and it has global scope. For all the other literals, they tend to be a `prvalue` and have statement scope.

	movable	unmovable
named	xvalue	lvalue
unnamed	prvalue	?

Throughout this paper, I have shown that it makes sense for temporaries [references and pointers] should have variable scope, unless they can be made global scope. From the programmers perspective, temporaries are just anonymously named variables. When they are passed as arguments, they have life beyond the life of the function that it is given to. As such the expression is not movable. As such, the desired behavior described throughout the paper is that they are `lvalues` which makes sense from a anonymously named standpoint. However, it must be said that technically they are unnamed which places them into the value category that `c++` currently does not have; the unmovable unnamed. The point is, this is simple whether it is worded as a `lvalue` or an unambiguous new value category that behaves like a `lvalue`. Regardless of which, there are some advantages that must be pointed out.

Avoids superfluous moves

The proposed avoids superfluous moves. Copying pointers and `lvalue` references are cheaper than performing a move which is cheaper than performing any non trivial value copy.

Undo forced naming

The proposed makes using types that delete their `rvalue` reference constructor easier to use. For instance, `std::reference_wrapper` can not be created/reassigned with a `rvalue` reference, i.e. temporaries. Rather, it must be created/reassigned with a `lvalue` reference created on a seperate line. This requires superfluous naming which increases the chances of dangling. Further, according to the `c++ Core Guidelines`, it is developers practice to do the following:

- *ES.5: Keep scopes small* [^{cppcg5}]
- *ES.6: Declare names in for-statement initializers and conditions to limit scope* [^{cppcg6}]

```

// currently not permitted; works as proposed
std::reference_wrapper<int> rwi1(5);
// current forced usage
int value1 = 5;
std::reference_wrapper<int> rwi2(value1);
if(randomBool())
{
    int value2 = 7;// ES.5, ES.6
    rwi2 = ref(value2);// dangles with block scope
    rwi2 = ref(7);// ok, safe and easy with variable scope proposal
    rwi2 = 7;// might make sense to add back direct assignment operator
}
else
{
    int value3 = 9;// ES.5, ES.6
    rwi2 = ref(value3);// dangles with block scope
    rwi2 = ref(9);// ok, safe and easy with variable scope proposal
    rwi2 = 9;// might make sense to add back direct assignment operator
}

```

Since the variable `value2` and `value3` is likely to be created manually at block scope instead of variable scope, it can accidentally introduce more dangling. Constructing and reassigning with a variable scoped `lvalue` temporary avoids these common dangling possibilities along with simplifying the code.

Consider too another example of forced naming.

```

int do_something_with_ref(int& i)
{
    return i;
}

int main()
{
    // clang
    // error: no matching function for call to 'do_something_with_ref'
    // note: candidate function not viable: expects an lvalue for 1st argument
    // msvc
    // error C2664: 'int do_something_with_ref(int &)': cannot convert argument 1
    // gcc
    // error: cannot bind non-const lvalue reference of type 'int&' to an rvalue
    return do_something_with_ref(0);
}

```

The previous code fails because the `do_something_with_ref` function is expecting a `lvalue`. However, the literal `0` is an `rvalue` when the temporary is scoped to the statement. This requires one of two possibilities, either the library writer has to overload the function such that `i` is `int&&` or library user has to name the variable.

library writer overloads method

```
int do_something_with_ref(int& i)
{
    return i;
}

int do_something_with_ref(int&& i)
{
    return i;
}

int main()
{
    return do_something_with_ref(0);
}
```

or

library user names the temporary

```
int do_something_with_ref(int& i)
{
    return i;
}

int main()
{
    int result = 0;
    return do_something_with_ref(result);
}
```

Templating the `do_something_with_ref` function with a universal reference would save the library writer from having to write the function twice but even that is an added complication.

library writer templatize method with universal reference

```

template<typename T>
T do_something_with_ref(T&& i)
{
    return i;
}

int main()
{
    return do_something_with_ref(0);
}

```

However, if the temporary `0` was scoped to the block and anonymously named than it would no longer be a `rvalue` and instead would be a `lvalue`.

```

int do_something_with_ref(int& i)
{
    return i;
}

int main()
{
    return do_something_with_ref(0);
}

```

No templating needed. No duplicate functions. No superfluous naming. Just more anonymous and concise, easy to understand code.

Allows more anonymous variables

The C++ Core Guidelines [^cppcgcp44] encourages programmers “to name your `lock_guards` and `unique_locks`” because “a temporary” “immediately goes out of scope”.

- *CP.44: Remember to name your `lock_guards` and `unique_locks`* [^cppcgcp44]


```

// useless when statement scoped
unique_lock<mutex>(m1);
lock_guard<mutex> {m2};
// current recommended usage
unique_lock<mutex> ul(m1);
lock_guard<mutex> lg{m2};
// even more useful when variable or block scoped
// this behaves the same had it been named
// one less name to return or worry about
unique_lock<mutex>(m1);
lock_guard<mutex> {m2};

```

With this proposal these instances do not immediately go out of scope. As such we get the locking benefits without having to make up a name. Again, not having a name means there is less to return and potentially dangle.

Summary

The advantages to C++ with adopting this proposal is manifold.

- Safer
 - Eliminate immediate dangling
 - Eliminate non return direct dangling that can occur in the body of a function
 - Eliminate returning indirect reference dangling when the instance was provided as an argument from the caller's scope
 - Reduce all remaining dangling
 - Reduce uninitialized and delayed initialization errors
- Simpler
 - Encourages the use of temporaries
 - Reduce lines of code
 - Reduce naming; fewer names to return dangle
 - Increases anonymously named `lvalues` and decreases `rvalues` in the code.
 - Reduce lines of code
 - Reduce naming; fewer names to return dangle
 - Simplify C++ temporary lifetime extension rules
 - Reduce the gap between C++ and C99 compound literals
 - Improve the potential contribution of C++'s dangling resolutions back to C
 - Simplify the language to match existing practice
 - Consequently, a “cleanup”, i.e. adoption of simpler, more general rules/guidelines

Frequently Asked Questions

Is `variable_scope` easy to teach?

values	pointers with <code>c99</code> &	proposed <code>c++</code>
<pre>int i = 5; if(whatever) { i = 7; } else { i = 9; } // use i</pre>	<pre>int* i = &5; // or uninitialized if(whatever) { i = &7; } else { i = &9; } // use i</pre>	<pre>std::reference_wrapper< if(whatever) { i = std::ref(7); } else { i = std::ref(9); } // use i</pre>

In the `values` example, there is no dangling. Programmers trust the compiler to allocate and deallocate instances on the stack. They have to because the programmer has little to no control over deallocation. With the current `c++` statement scope rules or the `c99` block scope rule, both the `pointers` and `references` examples dangle. In other words, the compilers who are primarily responsible for the stack has rules that needlessly causes dangling and embarrassing worse, immediate dangling. This violates the programmer's trust in their compiler. Variable scope is better because it restores the programmer's trust in their compiler/language by causing temporaries to match the value semantics of variables. Further, it avoids dangling throughout the body of the function whether it is anything that introduces new blocks/scopes be that `if` , `switch` , `while` , `for` statements and the nesting of these constructs.

Won't this break a lot of existing code?

Little, if any. To the contrary, code that is broken is now fixed. Code that would be invalid is now valid, makes sense and can be rationally explained.

When programmers use temporaries, unnamed variables, instead of named variables, then they give up control of the initialization order.

“7.6.1.3 Function call [expr.call]”

*“8 The postfix-expression is sequenced before each expression in the expression-list and any default argument. The initialization of a parameter, including every associated value computation and side effect, is **indeterminately** sequenced with respect to that of any other parameter.”*

“[Note 8: All side effects of argument evaluations are sequenced before the function is entered (see 6.9.1). — end note]”

Consequently, this indeterminateness remains regards of whether the temporary was scoped to the statement or the block. In this proposal, while the point of creation remains the same, the point of deletion gets extended just enough to remove **immediate** dangling. Since the temporary variable is by definition unnamed, any chance of breakage is greatly minimized because other than the parameter it was directly passed to, nothing else has a reference to it.

The proposed lifetimes matches or exceeds those of the current temporary lifetime extension rules in `C++`. In all of the standard examples the temporary lifetime is extended to the assigned variable. However, in reality, since all of these initial assignments are to the block then the real lifetime is to the block that contains the expression that contains the temporary. This is the same as this proposal. However, when the initialization statement is passed directly as an argument of a function call than it is the current statement lifetime since the argument is the variable. In other words, no additional life was given. This proposal improves the consistency by giving these argument temporaries the same block lifetime in order to eliminate immediate dangling and reduce further dangling. This proposal examines the `?:` ternary operator example and asks the question shouldn't this work for variables that are delayed initialized or reinitialized inside of the `if` and `else` clauses of `if/else` statements? Shouldn't the block of the variable declared above a `if/else` statement be used over the blocks of `if` and `else` clause where the temporary was assigned? This seems reasonable and would fix even more dangling. Consequently, all direct dangling is fixed within any given function and all we are left with are the indirect dangling which tends to be more complicated and easily avoided with simpler code.

Who would even use these features? Their isn't sufficient use to justify these changes.

Everyone ... Quite a bit, actually

Consider all the examples littered throughout our history, these are what gets fixed.

- dangling reported on normal use of the STL
- dangling examples reported in the C++ standard
- real world dangling reported in NAD, not a defect, reports

This doesn't even include the countless examples found in numerous articles comparing C++ with other nameless programming languages which would be fixed.

One of biggest gripes with the `Bind Returned/Initialized Objects to the Lifetime of Parameters` ^[4:1] proposal was that it would require a viral attribution effort. While that may be inevitable in order to fix indirect dangling in the language, that viral effort helps to identify the magnitude of C++ STL functions that has the potential of dangling in the first place and with this feature would no longer immediately dangle which is the most shocking type of dangling to end programmers.

Why not just use a static analyzer?

Typically a static analyzer doesn't fix code. Instead, it just produces warnings and errors. It is the programmer's responsibility to fix code by deciding whether the incident was a false positive or not and making the corresponding code changes. This proposal does fix most direct dangling but other, more indirect dangling goes unresolved and unidentified. As such this proposal and static analyzers are complimentary. Combined this proposal can fix most dangling and a static analyzer could be used to identify what is remaining. As such those who still ask, "why not just use a static analyzer", might really be saying **this proposal's language enhancements might break their static analyzer**. To which I say, the standard dictates the analyzer, not the other way around. That is true for all tools. However, let's explore the potential impact of this proposal on static analyzers.

The C++ language is complex. It stands to reason that our tools would have some degree of complexity, since they would need to take some subset of our language's rules into consideration. In any proposal, mine included, fixes to any dangling would result in potential dangling incidents becoming false positives between those identified by a static analyzer that overlap with said proposal. The false positives would join those that a static analyzer already has for not factoring existing language rules into consideration just as it would for any new language rules.

Static analyzers need to understand the lifetimes of variables with automatic storage duration, regardless. Not quantifying the current life of any given instance and determining whether it even needs to be extended would result in false positives. This already requires tracking braces/blocks/scopes. As such, tracking the statement that contains a temporary is not significantly more complicated than tracking the block that contains said expression and

temporary. In all likelihood, that is already being performed. Further, the proposed rules are significantly simpler than the current rules. This was identified by the numerous removals in the “Proposed Wording” section. That’s why this proposal would actually aid static analyzers.

Can this even be implemented?

C++ is already doing this for variable and for some instances of temporaries. What is proposed is that all temporaries should consistently benefit from this feature and even for temporaries to be made consistent with variables.

This proposal is simple. A compiler already knows when any variable is destroyed. So for this feature to work the compiler just needs an index on a per function basis to allow looking up the point of variable destruction given the variable again assuming this isn’t already a part of the variable metadata in the compiler. Best case this is a member access. Worst case it is a map.

This cost, additional or not, pails in comparison to proposals that fix dangling generally. Those have quadratic or exponential costs resolving variable dependency graphs. So it is a little hard to object to this proposal’s cost without swearing off fixing dangling in the language altogether. Further, C++ is already doing something similar with the `?:` ternary temporary lifetime extension example; 6.7.7 *Temporary objects* ^[10:2].

Doesn’t this make C++ harder to teach?

Until the day that all dangling gets fixed, any incremental fixes to dangling still would require programmers to be able to identify any remaining dangling and know how to fix it specific to the given scenario, as there are multiple solutions. Since dangling occurs even for things as simple as constants and immediate dangling is so naturally easy to produce, dangling resolution still have to be taught, even to beginners. As this proposal fixes a lot of dangling, it makes teaching C++ easier because it makes C++ easier.

So, what do we teach now and what bearing does these teachings, the C++ standard and this proposal have on one another.

C++ Core Guidelines

F.42: Return a `T*` to indicate a position (only) ^[7:1]

Note Do not return a pointer to something that is not in the caller’s scope; see F.43. ^[8:1]

Returning references to something in the caller’s scope is only natural. It is a part of our reference delegating programming model. A function when given a reference does not know how the instance was created and it doesn’t care as long as it is good for the life of the function call (and beyond). Unfortunately, scoping temporary arguments to the statement instead of the containing block doesn’t just create immediate dangling but it provides to functions references to instances that are near death. These instances are almost dead on arrival. Having the ability

to return a reference to a caller's instance or a sub-instance thereof assumes, correctly, that reference from the caller's scope would still be alive after this function call. The fact that temporary rules shortened the life to the statement is at odds with what we teach. This proposal restores to temporaries the lifetime of anonymously named variables which is not only natural but also consistent with what programmers already know. It is also in line with what we teach as was codified in the C++ Core Guidelines.

Other types of dangling can still occur. One simple type is directly called out in the C++ Core Guidelines.

C++ Core Guidelines

F.43: Never (directly or indirectly) return a pointer or a reference to a local object ^[8:2]

Reason *To avoid the crashes and data corruption that can result from the use of such a dangling pointer.* ^[8:3]

This proposal does not solve nor contradict this teaching. If anything, by cleaning up the other dangling it makes the remaining more visible. Also by hollowing out the majority and most common dangling in the middle, programmers are left with only indirect variants of the ultra trivial, such as indirect F.43 ^[8:4], or the ultra rare and ultimately more complex dangling, which is naturally avoided by keeping one's code simple.

Further, what is proposed is easy to teach because we already teach it and it makes c++ even easier to teach.

- We already teach RAI and that local variables are scoped to the block that contains them. This proposal just extends the concept to temporaries. This increases good consistency and removes or reduces multiple bifurcations that are currently taught;
 - that certain temporaries gets extended when assigned to a block but not when assigned to a argument
 - that variables and temporaries are all that different
 - that named and unnamed variables are different
- Somehow, we already teach the temporary lifetime extension rules which consist of numerous paragraphs and exception examples. These get replaced or greatly reduced to a few lines of verbage derived from c 's rule, which is only a couple of sentences.

All of this can be done without adding any new keywords or any new attributes. We just use variable concepts that beginners are already familiar with.

References

1. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2658r0.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2658r0.html>) ↔ ↔
2. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2623r2.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2623r2.html>) ↔
3. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2266r3.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2266r3.html>) ↔ ↔ ↔
4. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0936r0.pdf> (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0936r0.pdf>) ↔ ↔
5. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2012r2.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2012r2.pdf>) ↔ ↔ ↔ ↔ ↔ ↔ ↔ ↔ ↔ ↔
6. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2644r0.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2644r0.pdf>) ↔ ↔
7. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f42-return-a-t-to-indicate-a-position-only> (<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f42-return-a-t-to-indicate-a-position-only>) ↔ ↔
8. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f43-never-directly-or-indirectly-return-a-pointer-or-a-reference-to-a-local-object>
(<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f43-never-directly-or-indirectly-return-a-pointer-or-a-reference-to-a-local-object>) ↔ ↔ ↔ ↔ ↔
9. <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2731.pdf> (<https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2731.pdf>) ↔ ↔
10. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/n4910.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/n4910.pdf>) ↔ ↔ ↔